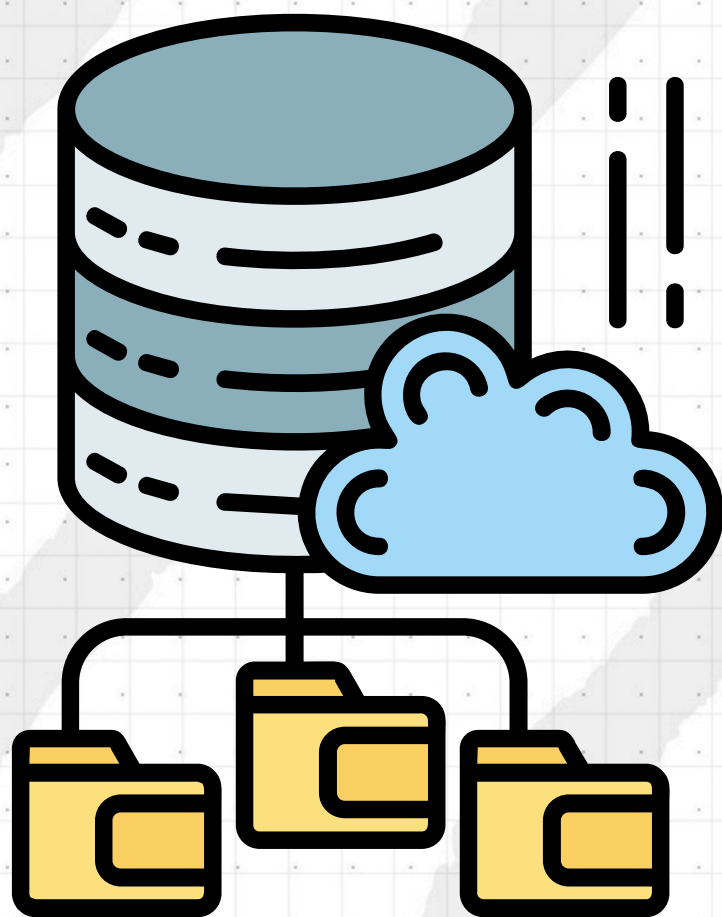


Top 30 SQL Window Functions Cheat Sheet



01

ROW_NUMBER()

Assigns a unique serial number to each row inside a group, useful to pick latest/first record.

ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY created_at)

02

RANK()

Ranks rows within a group; ties get same rank but next rank is skipped (1,1,3...).

RANK() OVER (PARTITION BY dept ORDER BY salary DESC)

03

ROW_NUMBER()

Ranks rows within a group; ties get same rank but ranks remain continuous (1,1,2...).

DENSE_RANK() OVER (ORDER BY score DESC)

04

NTILE(n)

Splits ordered data into N buckets (quartiles/deciles) to categorize segments like top 10%

NTILE(10) OVER (ORDER BY revenue DESC)

05

LAG(col, n)

Returns value from a previous row, used for day-over-day change, trend comparisons, churn detection.

LAG(sales, 1) OVER (PARTITION BY store ORDER BY day)

06

LEAD(col, n)

Returns value from a future row, useful for forecasting-style comparisons or next-step behavior analysis.

LEAD(sales, 1) OVER (PARTITION BY store ORDER BY day)

07

FIRST_VALUE()

Fetches the first value in a window, useful for baseline comparisons (first purchase, first status).

FIRST_VALUE(price) OVER (PARTITION BY product ORDER BY day)

08

LAST_VALUE()

Fetches the last value in a window; often used to compare current vs latest but needs correct frame.

LAST_VALUE(price) OVER (PARTITION BY product ORDER BY day)

09

NTH_VALUE(col, n)

Returns the nth record in a window, helpful for finding 2nd/3rd highest value or early milestones.

NTH_VALUE(salary, 3) OVER (PARTITION BY dept ORDER BY salary DESC)

10

SUM() OVER

Calculates running totals or group totals without collapsing rows like GROUP BY.

SUM(amount) OVER (PARTITION BY user_id ORDER BY day)

11

AVG() OVER

Computes moving averages or group averages; great for smoothing trends and monitoring performance.

AVG(sales) OVER (PARTITION BY store ORDER BY day)

12

COUNT() OVER

Counts rows per group while keeping row-level detail; useful for frequency analysis and totals.

COUNT(*) OVER (PARTITION BY category)

13

MIN() OVER

Returns minimum value inside a group/window, used for first date, lowest cost, earliest activity.

MIN(price) OVER (PARTITION BY product)

14

MAX() OVER

Returns maximum value inside a group/window, used for latest date, highest score, peak revenue.

MAX(score) OVER (PARTITION BY team)

15

STDDEV_POP() OVER

Measures population standard deviation in a group to understand variability and stability.

STDDEV_POP(sales) OVER (PARTITION BY store)

16

STDDEV_SAMP() OVER

Measures sample standard deviation, commonly used in analytics when data is a subset of full population.

STDDEV_SAMP(sales) OVER (PARTITION BY store)

17

VAR_POP() OVER

Population variance, useful to track volatility and detect abnormal fluctuations in a metric.

VAR_POP(sales) OVER (PARTITION BY store)

18

VAR_SAMP() OVER

Sample variance, used when analyzing variability from sampled data or incomplete datasets.

VAR_SAMP(sales) OVER (PARTITION BY store)

19

CUME_DIST()

Shows how far a row is in distribution; returns percentile position based on cumulative ranking.

CUME_DIST() OVER (ORDER BY salary)

20

PERCENT_RANK()

Shows relative rank between 0 and 1, helpful to identify top performers by percentile ranking.

PERCENT_RANK() OVER (ORDER BY score)

21

PERCENTILE_CONT(p)

Returns exact percentile value (continuous); great for median and p90 latency calculations.

**PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY salary)
OVER (PARTITION BY dept)**

22

PERCENTILE_DISC(p)

Returns nearest real value at percentile (discrete); useful when percentile must be an existing value.

**PERCENTILE_DISC(0.9) WITHIN GROUP (ORDER BY salary)
OVER ()**

23

MEDIAN() (Snowflake/Oracle)

Returns the middle value of distribution, more robust than AVG for skewed datasets.

MEDIAN(salary) OVER (PARTITION BY dept)

24

RATIO_TO_REPORT(col) (Oracle/Snowflake)

Calculates each row's share of the total, great for contribution analysis (e.g., % revenue by product).

RATIO_TO_REPORT(revenue) OVER (PARTITION BY region)

25

MODE() (DB specific)

Returns most frequent value in a group, useful for finding most common category/status.

**MODE() WITHIN GROUP (ORDER BY category) OVER (PARTITION
BY dept)**



26

APPROX_QUANTILES() (BigQuery)

Fast percentile estimation for massive datasets, perfect when exact percentiles are expensive.

```
APPROX_QUANTILES(score, 100)[OFFSET(90)] OVER ()
```

27

COUNTIF() OVER (BigQuery)

Count only rows that match a condition, useful for error rate, success/failure analysis.

```
COUNTIF(status='failed') OVER (PARTITION BY job)
```

28

QUALIFY (Snowflake/BigQuery)

Filters results after window functions, extremely useful for latest row per group in one query.

```
SELECT * FROM orders QUALIFY ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY created_at DESC) = 1;
```

29

ROWS BETWEEN ... (Frame)

Defines rolling windows by row count (e.g., last 7 rows), used in rolling averages & trend tracking.

```
SUM(sales) OVER ( PARTITION BY store ORDER BY day ROWS BETWEEN 6 PRECEDING AND CURRENT ROW )
```

30

RANGE BETWEEN ... (Frame)

Defines rolling windows by value range (e.g., last 7 days), perfect for time-based rolling metrics.

```
SUM(amount) OVER ( ORDER BY created_at RANGE BETWEEN INTERVAL '7' DAY PRECEDING AND CURRENT ROW)
```

